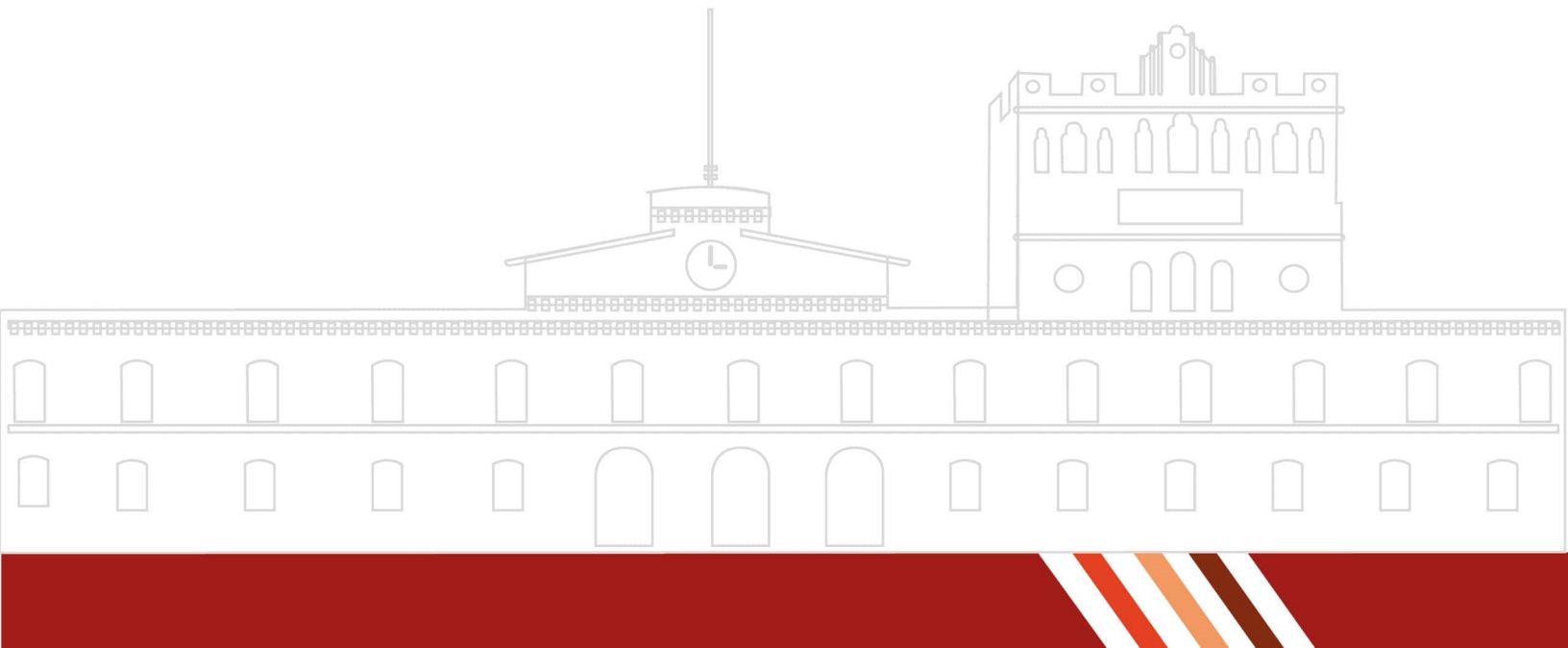


REPORTE DE PRÁCTICA NO. 2

RECONOCIMIENTO DE PALABRAS CON AUTÓMATAS FINITOS DETERMINISTAS

ALUMNO: Samuel Ortiz Salinas

Dr. Eduardo Cornejo-Velázquez



1. Identificación

- Nombre de la práctica: Reconocimiento de palabras con Autómatas Finitos Deterministas
- No. de práctica: 2
- No. de sesiones: 2
- No. de integrantes máximo por equipo: 2

2. Introducción

La teoría de autómatas estudia dispositivos de cálculo abstractos, es decir, máquinas capaces de exhibir conductas predeterminadas a partir de los estímulos que reciben del exterior [3]. Dentro de esta teoría, los Autómatas Finitos Deterministas (AFD) constituyen uno de los modelos más simples y, a la vez, de mayor aplicación práctica: se emplean en el diseño y prueba de circuitos digitales, en el analizador léxico de los compiladores y en la búsqueda de patrones dentro de texto [3]. Su estudio se enmarca dentro de la teoría de los lenguajes formales, disciplina que se vio revolucionada a partir de 1956 con los trabajos de Noam Chomsky sobre gramáticas transformacionales, los cuales sentaron las bases para formalizar los lenguajes de computación y las máquinas abstractas capaces de reconocerlos [1]. En esta práctica se construyen distintos AFD capaces de reconocer palabras pertenecientes a lenguajes definidos sobre diferentes alfabetos, mediante la definición de su tupla formal, su tabla de transiciones, su diagrama de estados y su simulación.

3. Marco teórico

Un **Autómata Finito Determinista (AFD)** se define como una tupla:

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- Σ es el alfabeto de entrada
- Q es el conjunto finito y no vacío de los estados
- δ es la función de transición, $\delta : Q \times \Sigma \rightarrow Q$
- $q_0 \in Q$ es el estado inicial
- $F \subset Q$ es un conjunto de estados finales de aceptación ($F \neq \emptyset$)

Sea $A = (\Sigma, Q, \delta, q_0, F)$ un AFD y sea $w = w_1 w_2 \dots w_n$ una cadena de símbolos donde $w_i \in \Sigma$. Entonces, A **acepta** w si existe una secuencia de estados $r_0, r_1, \dots, r_n \in Q$ con tres condiciones:

1. $r_0 = q_0$
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ para $i = 0, 1, \dots, n-1$
3. $r_n \in F$

Entonces, A **reconoce el lenguaje** L si $L = \{w \mid A \text{ acepta } w\}$.

Una **tabla de transición de estados** es una tabla que muestra a qué estado se moverá un autómata finito, basándose en el estado actual y las entradas.

Generalmente se asocia con cada autómata un **diagrama de transición de estados**. Cada nodo del diagrama corresponde a un estado. El estado inicial se indica mediante una flecha que no tiene nodo origen. Los estados finales se representan con un círculo doble.

4. Objetivos

Objetivo general

Construir Autómatas Finitos Deterministas (AFD) para reconocer palabras de un lenguaje L mediante la definición de tablas de transiciones de estado, diagramas de transiciones de estado y simulaciones.

Objetivos específicos

- Implementar Autómatas Finitos Deterministas (AFD).
- Construir tablas de transiciones de estados.
- Construir diagramas de transiciones de estados.
- Realizar la simulación de AFD para validar palabras válidas y no válidas en el lenguaje L.

5. Equipo y software a utilizar

Equipo de cómputo con software para programación a elección de los alumnos. Simulación realizada en Automaton Simulator.

6. Desarrollo

Para cada ejercicio se define: la tupla $AFD = (\Sigma, Q, \delta, q_0, F)$, la tabla de transiciones de estados, el diagrama de transiciones de estados y la simulación con palabras aceptadas y rechazadas.

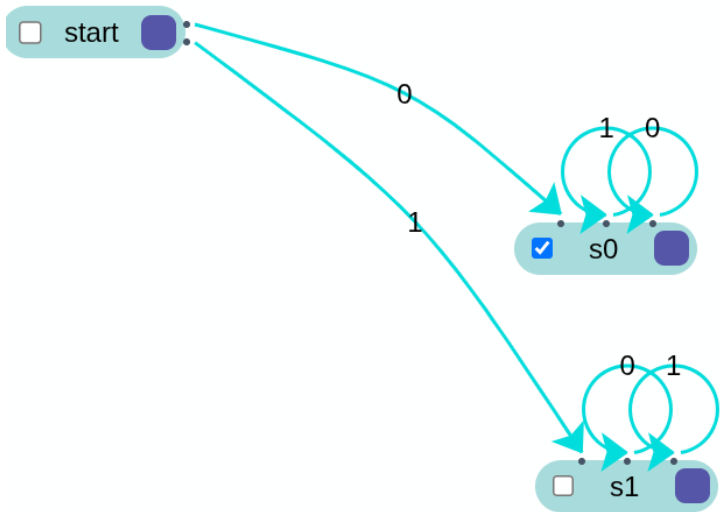
Ejercicio 1

$AFD = (\Sigma, Q, \delta, q_0, F)$

- $\Sigma = \{0, 1\}$
- $Q = \{start, s_0, s_1\}$
- $q_0 = start$
- $F = \{s_0\}$

Table 1: Tabla de transiciones de estados.

Estado	0	1
$\rightarrow start$	s_0	s_1
$*s_0$	s_0	s_0
s_1	s_1	s_1



Test Results:

Accept: 0 -- Pass
Accept: 00 -- Pass
Accept: 01 -- Pass
Accept: 010 -- Pass
Accept: 0110 -- Pass
Reject: 1 -- Pass
Reject: 10 -- Pass
Reject: 11 -- Pass
Reject: 100 -- Pass
Reject: 111 -- Pass

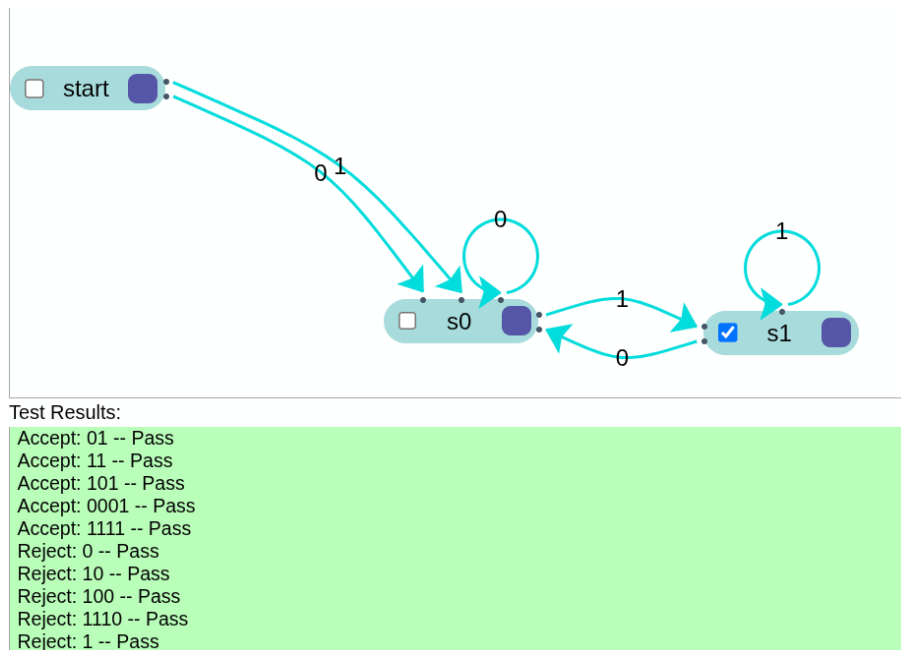
Ejercicio 2

$AFD = (\Sigma, Q, \delta, q_0, F)$

- $\Sigma = \{0, 1\}$
- $Q = \{start, s_0, s_1\}$
- $q_0 = start$
- $F = \{s_1\}$

Table 2: Tabla de transiciones de estados.

Estado	0	1
$\rightarrow start$	s_0	s_0
s_0	s_0	s_1
$*s_1$	s_0	s_1



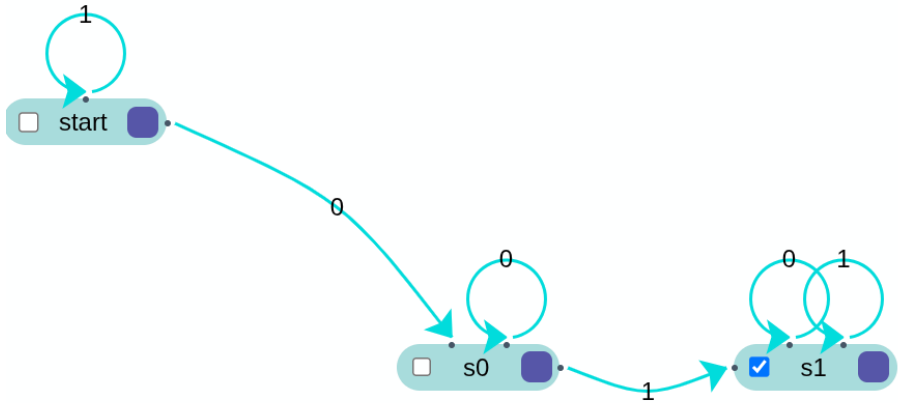
Ejercicio 3

$AFD = (\Sigma, Q, \delta, q_0, F)$

- $\Sigma = \{0, 1\}$
- $Q = \{start, s_0, s_1\}$
- $q_0 = start$
- $F = \{s_1\}$

Table 3: Tabla de transiciones de estados.

Estado	0	1
$\rightarrow start$	s_0	$start$
s_0	s_0	s_1
$*s_1$	s_1	s_1



Test Results:

Accept: 01 -- Pass
Accept: 001 -- Pass
Accept: 010 -- Pass
Accept: 1011 -- Pass
Accept: 0011 -- Pass
Reject: 0 -- Pass
Reject: 1 -- Pass
Reject: 00 -- Pass
Reject: 11 -- Pass
Reject: 000 -- Pass

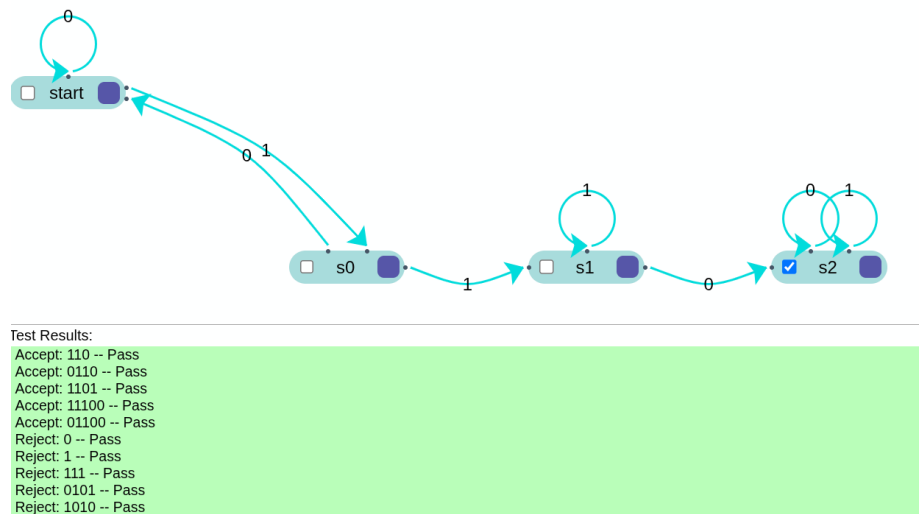
Ejercicio 4

$AFD = (\Sigma, Q, \delta, q_0, F)$

- $\Sigma = \{0, 1\}$
- $Q = \{start, s_0, s_1, s_2\}$
- $q_0 = start$
- $F = \{s_2\}$

Table 4: Tabla de transiciones de estados.

Estado	0	1
$\rightarrow start$	$start$	s_0
s_0	$start$	s_1
s_1	s_2	s_1
$*s_2$	s_2	s_2



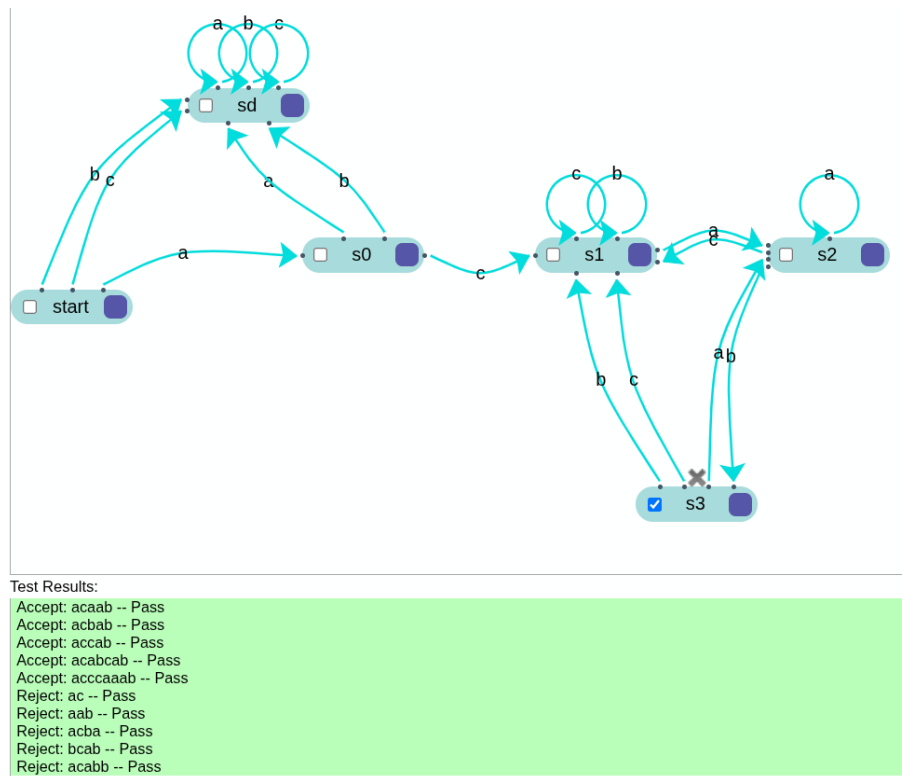
Ejercicio 5

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- $\Sigma = \{a, b, c\}$
- $Q = \{start, s_0, s_d, s_1, s_2, s_3\}$
- $q_0 = start$
- $F = \{s_3\}$

Table 5: Tabla de transiciones de estados.

Estado	a	b	c
$\rightarrow start$	s_0	s_d	s_d
s_0	s_d	s_d	s_1
s_d	s_d	s_d	s_d
s_1	s_2	s_1	s_1
s_2	s_2	s_3	s_1
$*s_3$	s_2	s_1	s_1



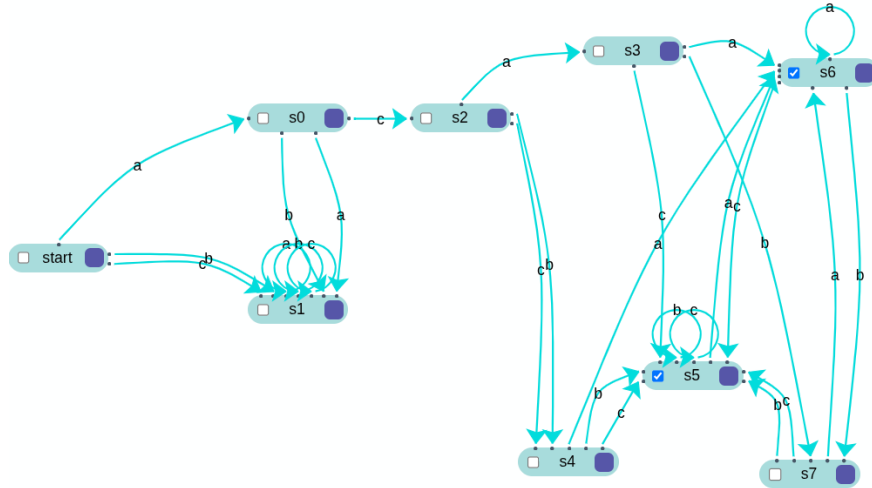
Ejercicio 6

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- $\Sigma = \{a, b, c\}$
- $Q = \{start, s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$
- $q_0 = start$
- $F = \{s_5, s_6\}$

Table 6: Tabla de transiciones de estados.

Estado	a	b	c
$\rightarrow start$	s_0	s_1	s_1
s_0	s_1	s_1	s_2
s_1	s_1	s_1	s_1
s_2	s_3	s_4	s_4
s_3	s_6	s_7	s_5
s_4	s_6	s_5	s_5
$*s_5$	s_6	s_5	s_5
$*s_6$	s_6	s_7	s_5
s_7	s_6	s_5	s_5



Test Results:
 Accept: acaa -- Pass
 Accept: acbb -- Pass
 Accept: accc -- Pass
 Accept: acbca -- Pass
 Accept: acaccb -- Pass
 Reject: acab -- Pass
 Reject: ac -- Pass
 Reject: abab -- Pass
 Reject: bcab -- Pass
 Reject: acbab -- Pass

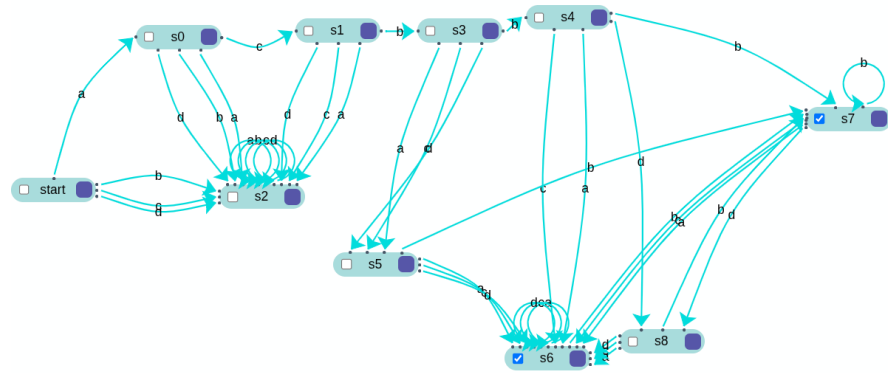
Ejercicio 7

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- $\Sigma = \{a, b, c, d\}$
- $Q = \{start, s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8\}$
- $q_0 = start$
- $F = \{s_6, s_7\}$

Table 7: Tabla de transiciones de estados.

Estado	a	b	c	d
$\rightarrow start$	s_0	s_2	s_2	s_2
s_0	s_2	s_2	s_1	s_2
s_1	s_2	s_3	s_2	s_2
s_2	s_2	s_2	s_2	s_2
s_3	s_5	s_4	s_5	s_5
s_4	s_6	s_7	s_6	s_8
s_5	s_6	s_7	s_6	s_6
$*s_6$	s_6	s_7	s_6	s_6
$*s_7$	s_6	s_7	s_6	s_8
s_8	s_6	s_7	s_6	s_6



Test Results:
 Accept: acbaa -- Pass
 Accept: acbcc -- Pass
 Accept: acbdd -- Pass
 Accept: acbca -- Pass
 Accept: acbdba -- Pass
 Reject: acbdd -- Pass
 Reject: acb -- Pass
 Reject: abcd -- Pass
 Reject: dcba -- Pass
 Reject: acbccbd -- Pass

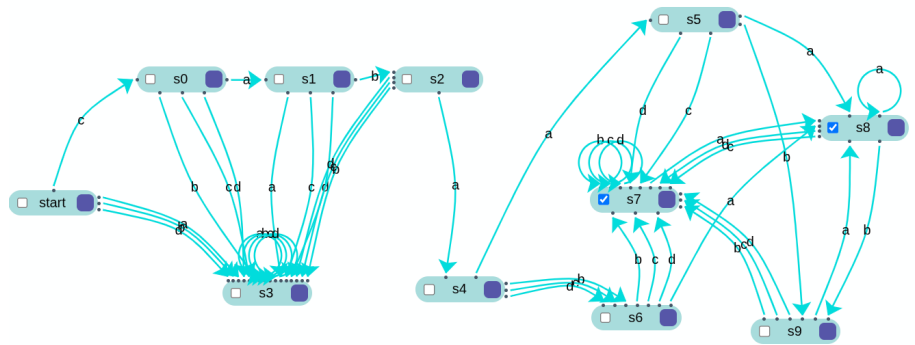
Ejercicio 8

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- $\Sigma = \{a, b, c, d\}$
- $Q = \{start, s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9\}$
- $q_0 = start$
- $F = \{s_7, s_8\}$

Table 8: Tabla de transiciones de estados.

Estado	a	b	c	d
$\rightarrow start$	s_3	s_3	s_0	s_3
s_0	s_1	s_3	s_3	s_3
s_1	s_3	s_2	s_3	s_3
s_2	s_4	s_3	s_3	s_3
s_3	s_3	s_3	s_3	s_3
s_4	s_5	s_6	s_6	s_6
s_5	s_8	s_9	s_7	s_7
s_6	s_8	s_7	s_7	s_7
$*s_7$	s_8	s_7	s_7	s_7
$*s_8$	s_8	s_9	s_7	s_7
s_9	s_8	s_7	s_7	s_7



Test Results:

```

Accept: cabaaa -- Pass
Accept: cabacc -- Pass
Accept: cabadd -- Pass
Accept: cababca -- Pass
Accept: cabadcb -- Pass
Reject: cababab -- Pass
Reject: cabab -- Pass
Reject: caba -- Pass
Reject: cab -- Pass
Reject: cab -- Pass
Reject: abca -- Pass
Reject: cababab -- Pass

```

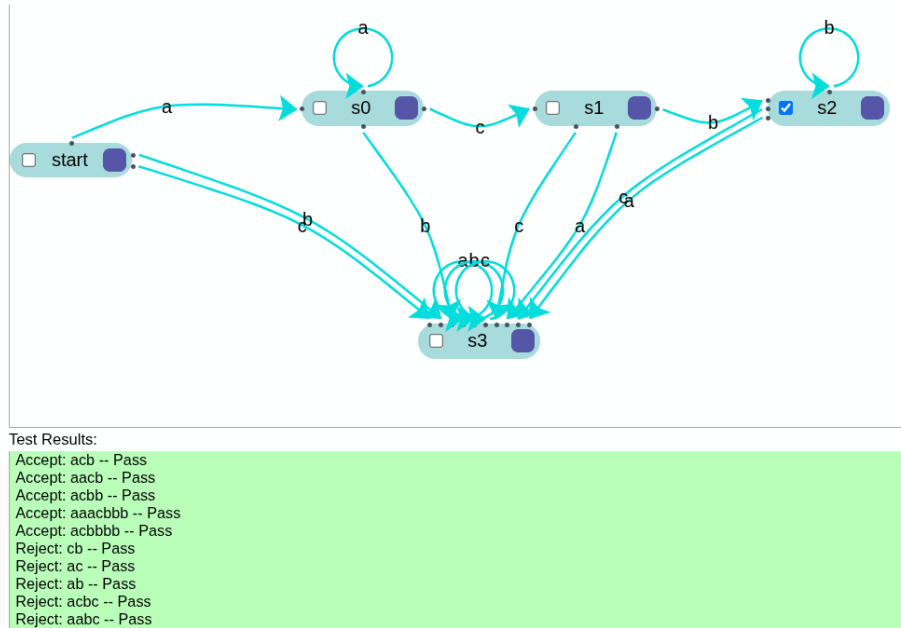
Ejercicio 9

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- $\Sigma = \{a, b, c\}$
- $Q = \{start, s_0, s_1, s_2, s_d\}$
- $q_0 = start$
- $F = \{s_2\}$

Table 9: Tabla de transiciones de estados.

Estado	a	b	c
$\rightarrow start$	s_0	s_d	s_d
s_0	s_0	s_d	s_1
s_1	s_d	s_2	s_d
$*s_2$	s_d	s_2	s_d
s_d	s_d	s_d	s_d



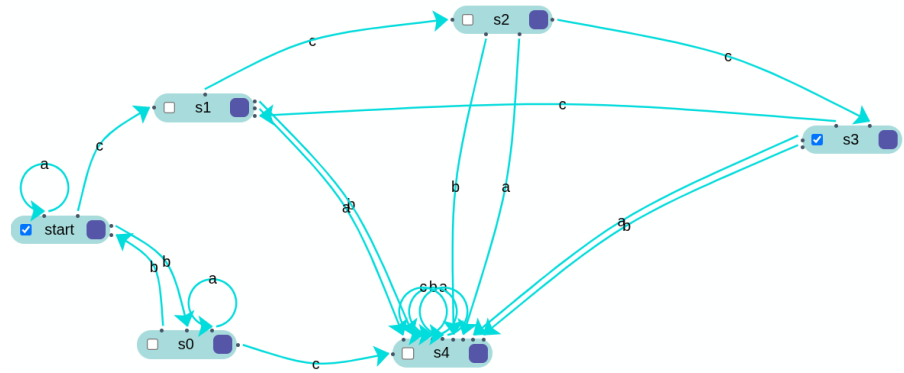
Ejercicio 10

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- $\Sigma = \{a, b, c\}$
- $Q = \{start, s_0, s_1, s_2, s_3, s_4\}$
- $q_0 = start$
- $F = \{start, s_3\}$

Table 10: Tabla de transiciones de estados.

Estado	a	b	c
$\rightarrow *start$	<i>start</i>	<i>s₀</i>	<i>s₁</i>
<i>s₀</i>	<i>s₀</i>	<i>start</i>	<i>s₄</i>
<i>s₁</i>	<i>s₄</i>	<i>s₄</i>	<i>s₂</i>
<i>s₂</i>	<i>s₄</i>	<i>s₄</i>	<i>s₃</i>
<i>*s₃</i>	<i>s₄</i>	<i>s₄</i>	<i>s₁</i>
<i>s₄</i>	<i>s₄</i>	<i>s₄</i>	<i>s₄</i>



Test Results:

Accept: a -- Pass
 Accept: bb -- Pass
 Accept: abab -- Pass
 Accept: accc -- Pass
 Accept: bbccccc -- Pass
 Reject: b -- Pass
 Reject: ac -- Pass
 Reject: abcc -- Pass
 Reject: bbcc -- Pass
 Reject: aabcccc -- Pass

7. Resultados

Se construyeron y formalizaron los AFD solicitados en el procedimiento de la práctica, cada uno con su tupla $(\Sigma, Q, \delta, q_0, F)$, su tabla de transiciones y su diagrama de estados. Todos los autómatas fueron simulados en Automaton Simulator, verificando en cada caso 5 palabras que pertenecen al lenguaje L (aceptadas) y 5 que no pertenecen (rechazadas), obteniendo en todos los casos el resultado esperado (*Pass*).

Table 11: Resumen de los AFD construidos.

Ejercicio	Alfabeto Σ	No. de estados
1	$\{0, 1\}$	3
2	$\{0, 1\}$	3
3	$\{0, 1\}$	3
4	$\{0, 1\}$	4
5	$\{a, b, c\}$	6
6	$\{a, b, c\}$	9
7	$\{a, b, c, d\}$	10
8	$\{a, b, c, d\}$	11
9	$\{a, b, c\}$	5
10	$\{a, b, c\}$	6

En los lenguajes que exigían un prefijo y un sufijo fijos sobre una parte arbitraria de la cadena (Ejercicios 5 a 8), el número de estados aumentó respecto de los lenguajes de "contiene" o "empieza/termina con" (Ejercicios 1 a 4), debido a la necesidad de combinar un autómata de reconocimiento de prefijo con un autómata de coincidencia de sufijo (tipo KMP) que maneja los posibles solapamientos entre símbolos.

8. Conclusiones

El desarrollo de esta práctica permitió reforzar la construcción de Autómatas Finitos Deterministas a partir de la definición formal de un lenguaje L , pasando de la expresión matemática del lenguaje a la tupla $(\Sigma, Q, \delta, q_0, F)$, su tabla de transiciones y su diagrama de estados. Mientras que los lenguajes con prefijo y sufijo fijos requieren combinar varios autómatas más simples (prefijo + patrón de sufijo con manejo de solapamientos) en uno solo. La simulación de cada AFD en Automaton Simulator resultó de gran ayuda para validar rápidamente el diseño antes de darlo por correcto, detectando de forma visual e inmediata cualquier error en las transiciones o en los estados de aceptación.

9. Cuestionario

1. ¿Cuáles son los elementos que definen un AFD?

Un AFD se define como la tupla $(\Sigma, Q, \delta, q_0, F)$: Σ es el alfabeto de entrada, Q el conjunto finito de estados, $\delta : Q \times \Sigma \rightarrow Q$ la función de transición, $q_0 \in Q$ el estado inicial y $F \subset Q$ es el conjunto de estados de aceptación.

2. ¿Cuál es la utilidad de una tabla de transiciones de estado de un AFD?

Permite representar de forma compacta y sin ambigüedad la función δ completa del autómata, mostrando para cada estado y cada símbolo del alfabeto cuál es el estado destino. Esto facilita tanto el diseño manual del AFD como su traducción directa a una estructura de datos (por ejemplo, una matriz) para implementarlo en un lenguaje de programación.

3. ¿Qué importancia tienen los diagramas de transiciones de estado en el proceso de construcción de un AFD?

El diagrama es la representación gráfica del autómata: cada nodo es un estado, cada arco etiquetado es una transición, el estado inicial se señala con una flecha de entrada y los estados de aceptación con un círculo doble. Ayuda a visualizar el comportamiento del autómata como un todo, a detectar errores de diseño (estados inalcanzables, transiciones faltantes) y a comunicar el modelo de forma clara antes de implementarlo.

4. ¿Cuáles son las ventajas de la simulación del AFD?

Permite validar de forma rápida y visual si el autómata reconoce correctamente el lenguaje L para el que fue diseñado, probando tanto palabras que deberían ser aceptadas como palabras que deberían ser rechazadas, sin necesidad de programar el AFD. Esto ayuda a detectar errores en la tupla o en la tabla de transiciones antes de pasar a una implementación real.

Referencias Bibliográficas

References

- [1] Giró, J., Vázquez, J., Meloni, B., Constable, L. **(2015)**. *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.
- [2] Ruiz Catalán, J. **(2010)**. *Compiladores: teoría e implementación*. Editorial Alfaomega. México.
- [3] Hopcroft, J. E., Motwani, R., Ullman, J. D. **(2007)**. *Introducción a la teoría de autómatas, lenguajes y computación*. Editorial Pearson Addison-Wesley. Tercera edición. Madrid, España.